

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	100
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	V ADARSH REDDY	Reg.No.:	192525412
Department:	B TECH AI/ML	Date:	18/08/26

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

EXPERIMENT 1 – REGISTER TRANSFER AND ALU OPERATIONS

Aim: To design and simulate a register-transfer system that performs register transfers together with addition, subtraction, AND and OR operations using multiple registers and an ALU.

Objectives

- Demonstrate data movement between registers.
- Implement arithmetic and logic operations through an ALU.
- Relate register-transfer operations to control signals and datapath behavior.
- Validate program results using multiple test cases.

Concept / Theory

A register transfer operation moves binary data from one register to another under the control of a control signal. The ALU performs arithmetic and logical functions on operands supplied by registers. A datapath connects registers, multiplexers and the ALU, while the control unit selects the required operation.

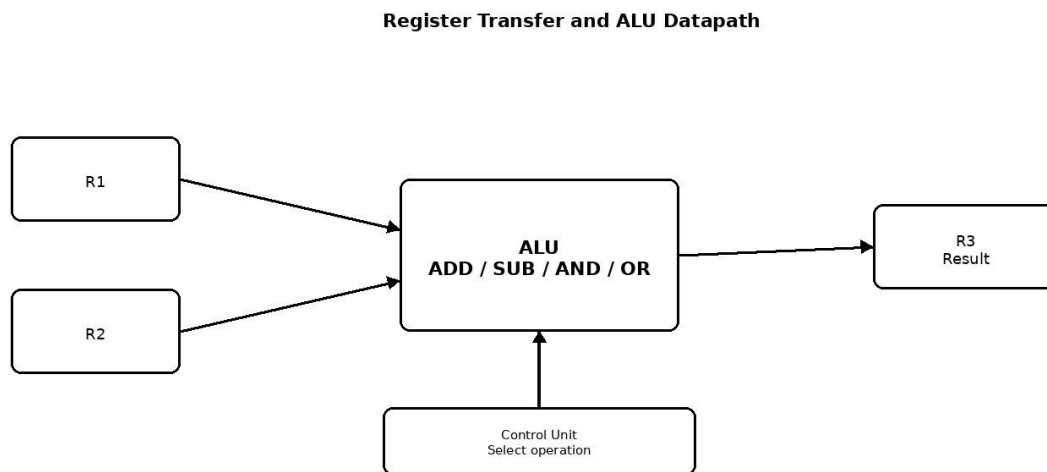


Figure 1. Register-transfer and ALU datapath.

Software / Tools Required

- GNUSim8085 – 8085 microprocessor simulator
- Logisim Evolution for optional circuit-level validation

Experimental Design

The 8085 registers B and C are loaded with 0FH and 05H. The accumulator performs ADD, SUB, AND and OR operations. Results are stored at 2000H–2003H and register transfers B→D and C→E are verified.

Test	R1	R2	Operation	Expected
T1	0FH	05H	ADD	14H (20 decimal)
T2	0FH	05H	SUB	0AH (10 decimal)
T3	0FH	05H	AND	05H (5 decimal)
T4	0FH	05H	OR	0FH (15 decimal)
T5	0FH	05H	MOV B→D, C→E	D=0FH, E=05H

Algorithm

- Read R1, R2 and operation.

- Select the corresponding ALU function.
- Compute and store the result in R3.
- Compare the computed result with the expected value.
- Display PASS/FAIL and the operation count.

Program

MVI B, 0FH

MVI C, 05H

; ADDITION

MOV A, B

ADD C

STA 2000H

; SUBTRACTION

MOV A, B

SUB C

STA 2001H

; AND

MOV A, B

ANA C

STA 2002H

; OR

MOV A, B

ORA C

STA 2003H

; REGISTER TRANSFER

MOV D, B

MOV E, C

HLT

Output to Capture:

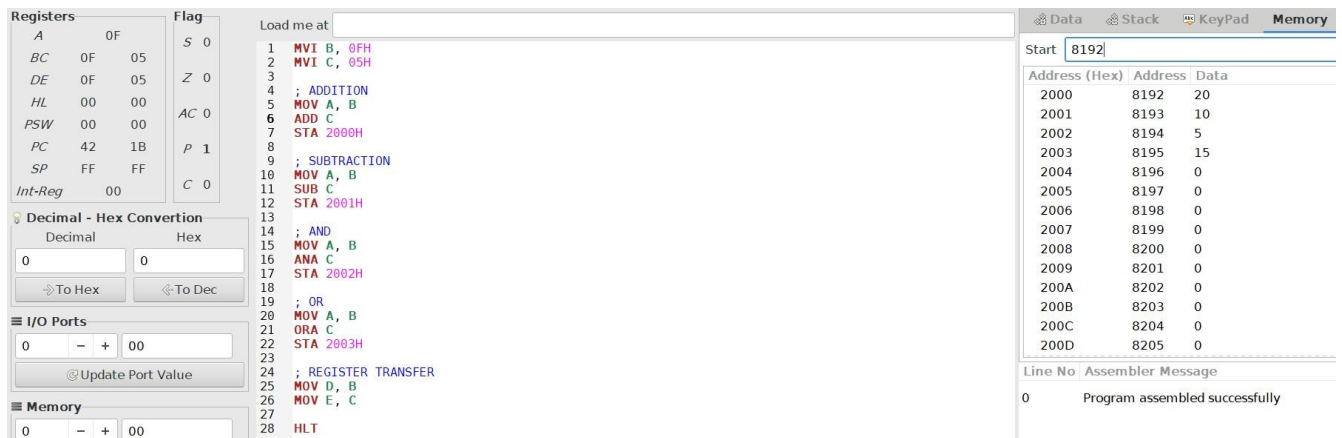


Figure 1A. GNUSim8085 memory output after execution.

Capture the GNUSim8085 program screen showing successful assembly and register values, and capture the Memory tab showing 2000H–2003H results. These screenshots are the experimental evidence.

Observed execution result: 2000H=14H, 2001H=0AH, 2002H=05H and 2003H=0FH. After execution, B=0FH, C=05H, D=0FH and E=05H.

Results and Analysis

Parameter	Observation to Record
Total valid test cases	Enter actual value
Passed cases	Enter actual value
Failed/rejected cases	Enter actual value
ALU operations demonstrated	ADD, SUB, AND, OR
Circuit validation	PASS/FAIL after Logisim comparison

Validation

Validation is performed by comparing the computed result with the expected arithmetic/bitwise result and, where Logisim is used, comparing the program result with the circuit output.

Result: The register-transfer and ALU operations are successfully simulated when all valid test cases produce PASS and the invalid operation is correctly rejected. IPC and resource utilization are measured and interpreted against the scalar baseline.

2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.

EXPERIMENT 2 – SINGLE-BUS AND MULTI-BUS ORGANIZATION

Aim: To model single-bus and multi-bus computer organizations, execute equivalent data-transfer sequences and compare execution time, clock cycles and efficiency.

Objectives

- Model a common-bus datapath and a multi-bus datapath.
- Perform register, memory and I/O-related transfers.
- Measure the number of transfer cycles required.
- Explain how parallel paths reduce bottlenecks.

Concept / Theory

In a single-bus organization, registers and functional units share one common communication path, so only one bus transfer can normally be performed at a time. A multi-bus organization provides multiple simultaneous paths, allowing operands to reach the ALU in parallel and reducing the number of clock cycles for suitable operations.

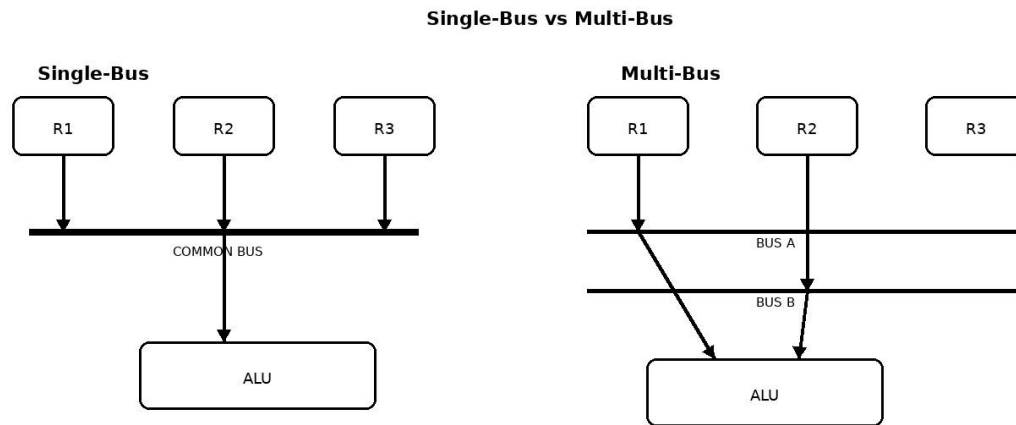


Figure 2. Conceptual comparison of single-bus and multi-bus organizations.

Software / Tools Required

- Logisim Evolution – primary circuit modeling tool
- Python 3.x – cycle and efficiency analysis

Experimental Design

Use the same logical sequence for both architectures. Record the number of bus transfers and the number of clock cycles. For a fair comparison, keep the operation sequence, data values and required final result identical.

Operation	Single-Bus Action	Multi-Bus Action
1	R1 → Bus → R2	R1 → Bus A → R2
2	R2 → Bus → R3	R2 → Bus A → R3
3	R3 and R4 to ALU	R3 on Bus A, R4 on Bus B
4	ALU result → R5	ALU result → R5
5	R5 → Memory	R5 → Memory

Logisim Implementation Checklist

- Place registers R1–R5.
- Build a common bus and tri-state/multiplexer selection.
- Connect the ALU and memory.

- Build the multi-bus version with two operand paths.
- Use identical input values and operation sequence.
- Record clock cycles from the simulator.

Output / Evidence

```
=====
                        PERFORMANCE ANALYSIS
=====

Single-Bus Cycles      : 6
Multi-Bus Cycles       : 5

Single-Bus Transfers   : 5
Multi-Bus Transfers    : 5

Speedup                : 1.20x
Cycle Reduction        : 16.67%
Single-Bus Efficiency  : 0.83
Multi-Bus Efficiency   : 1.00

Interpretation:
The multi-bus organization reduces execution cycles because two operands can be transferred to the ALU simultaneously using separate buses.

=====
                        VALIDATION
=====

Single-Bus final memory value : 15
Multi-Bus final memory value  : 15

VALIDATION : PASS
Both architectures produced the same final result.
```

Results and Analysis

Metric	Single Bus	Multi Bus	Improvement
Clock cycles	Actual	Actual	Calculate
Data transfers/cycle	Actual	Actual	Calculate
Execution time	Actual	Actual	Calculate
Speedup	1.00×	Actual	Calculate
Cycle reduction	0%	Actual	Calculate

Analysis: Explain which transfers are serialized in the single-bus design and which can overlap in the multi-bus design. Relate the measured reduction to parallel data movement and discuss the hardware/control complexity added by multiple buses.

Validation

Both architectures must produce the same final register/memory state for the same input sequence. Only the transfer organization and measured timing should differ.

Result

The single-bus and multi-bus organizations are successfully compared when the same logical task produces the same final state and the measured cycle counts demonstrate the effect of additional parallel data paths.

3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

EXPERIMENT 3 – FIVE-STAGE INSTRUCTION PIPELINE

Aim: To simulate a five-stage instruction pipeline consisting of Fetch, Decode, Execute, Memory and Write-back stages and compare its performance with a non-pipelined system.

Objectives

- Simulate instruction overlap across five stages.
- Generate a cycle-by-cycle pipeline schedule.
- Measure speedup and throughput.
- Validate the result against the theoretical pipeline-cycle formula.

Concept / Theory

Instruction pipelining overlaps the execution of multiple instructions by dividing instruction processing into stages. The five stages used here are IF, ID, EX, MEM and WB. After the pipeline is filled, ideally one instruction completes per cycle.

Five-Stage Instruction Pipeline

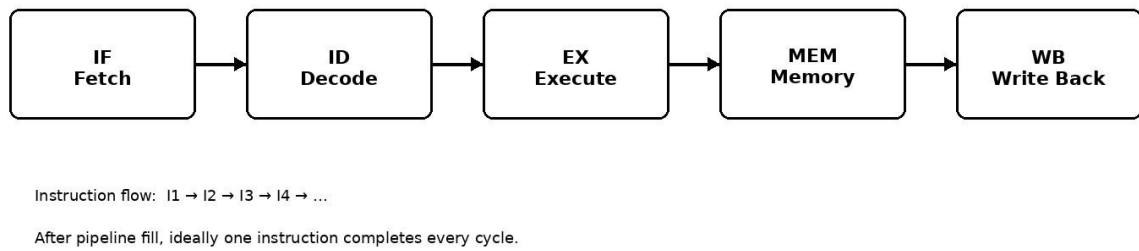


Figure 3. Five-stage instruction pipeline.

Experimental Design

The experiment uses N independent instructions. A non-pipelined model completes every instruction through all five stages before starting the next one. The pipelined model overlaps stages. The ideal pipelined cycle count for N instructions and k stages is $N + k - 1$.

Test Cases

Case	Number of Instructions	Purpose
C1	5	Basic pipeline fill and completion
C2	10	Observe better amortization of pipeline fill
C3	20	Compare throughput for a longer instruction stream

To test C2 and C3, replace the instruction list with 10 and 20 instruction labels and run the same program

.

Output / Evidence

Cycle-by-cycle pipeline schedule:						
Cycle	I1	I2	I3	I4	I5	I6
1	IF	-	-	-	-	-
2	ID	IF	-	-	-	-
3	EX	ID	IF	-	-	-
4	MEM	EX	ID	IF	-	-
5	WB	MEM	EX	ID	IF	-
6	-	WB	MEM	EX	ID	IF
7	-	-	WB	MEM	EX	ID
8	-	-	-	WB	MEM	EX
9	-	-	-	-	WB	MEM
10	-	-	-	-	-	WB
Total pipelined cycles : 10						
=====						
PERFORMANCE ANALYSIS						
=====						
Non-pipelined cycles : 30						
Pipelined cycles : 10						
Speedup : 3.00x						
Cycle reduction : 66.67%						
Pipeline throughput : 0.600 instructions/cycle						
Pipeline efficiency : 60.00%						
=====						
VALIDATION						
=====						
Measured pipeline cycles : 10						
Theoretical pipeline cycles : 10						
VALIDATION : PASS						
The simulated pipeline cycle count matches the theoretical value.						

Analysis and Validation

Compare the measured program values with the theoretical equations. For $k=5$ stages, the ideal pipelined cycle count is $N+4$. The non-pipelined count is $5N$. Speedup should approach the number of stages as N becomes large, assuming no hazards and equal stage times.

Result

The five-stage pipeline is successfully simulated when the cycle schedule, theoretical cycle count, speedup and throughput are consistent.

4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

EXPERIMENT 4 – PIPELINE HAZARDS AND THEIR RESOLUTION

Aim: To develop pipeline scenarios containing data, control and structural hazards and evaluate stalling, data forwarding and branch prediction as hazard-resolution techniques.

Objectives

- Identify data, control and structural hazards.
- Measure the performance cost of hazards.
- Compare stalling with forwarding and branch prediction.
- Interpret the effect of each technique on execution time.

Concept / Theory

A data hazard occurs when an instruction depends on a value that has not yet been written back. A control hazard is caused by uncertainty in the next instruction due to branches. A structural hazard occurs when two instructions need the same hardware resource at the same time. Forwarding reduces data-hazard stalls, branch prediction reduces control-hazard penalties, and stalling or resource scheduling resolves structural conflicts.

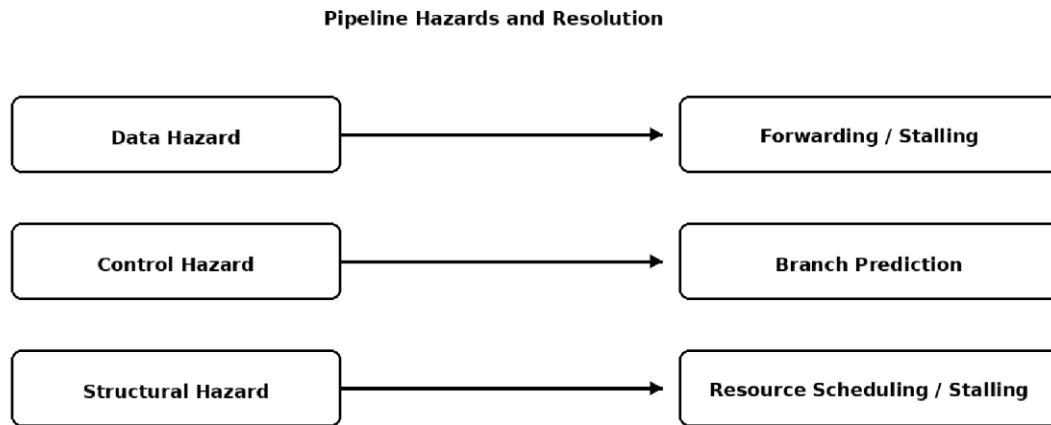


Figure 4. Pipeline hazards and representative resolution techniques.

Experimental Design

Three instruction streams are evaluated: a data-dependent sequence, a branch sequence and a resourceconflict sequence. Each scenario is evaluated in a baseline mode and an optimized mode. The program reports cycles and stalls.

Important Note on the Model

The program uses an explicit simplified timing model so that each hazard and its resolution can be observed independently. For a more detailed implementation, the same scenarios can be built as a cycle-by-cycle simulator with individual IF, ID, EX, MEM and WB states.

Output / Evidence

```
--- Without Resource Scheduling ---
Resource conflict occurs.
Stalls introduced : 2
Total cycles      : 8

--- With Resource Scheduling ---
Instructions are scheduled so that the resource is not requested simultaneously.
Stalls introduced : 0
Total cycles      : 6

Speedup with scheduling : 1.33x

=====
                        PERFORMANCE SUMMARY
=====

Hazard                Baseline    Optimized    Speedup
-----
Data Hazard           7          5          1.40x
Control Hazard        8          6          1.33x
Structural Hazard     8          6          1.33x

=====
                        VALIDATION
=====

Data Hazard:
Forwarding removes unnecessary waiting.
VALIDATION : PASS

Control Hazard:
Branch prediction reduces branch penalty.
VALIDATION : PASS

Structural Hazard:
Resource scheduling prevents resource conflicts.
VALIDATION : PASS

Overall Validation : PASS
```

Results and Analysis

Hazard	Baseline Effect	Resolution	What to Interpret
Data	Extra stall cycles	Forwarding	Reduced waiting for dependent data
Control	Branch penalty	Prediction	Reduced wrong-path waiting

Structural	Resource conflict	Scheduling/stall	Prevents simultaneous resource use
------------	-------------------	------------------	------------------------------------

Analysis: Compare baseline and optimized cycle counts. Explain why forwarding can reduce a data dependency without waiting for write-back, why prediction can reduce branch delay, and why structural hazards require resource-aware scheduling or stalls.

Validation

Verify that optimized execution produces the same intended instruction results while using fewer or equal cycles. The performance improvement must not come from removing required instructions.

Result

The three major pipeline hazards are successfully demonstrated and their resolution techniques are compared using measurable cycle and stall counts.

5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

EXPERIMENT 5 – SUPERSCALAR, OUT-OF-ORDER AND SMT EXECUTION

Aim: To analyze a processor supporting superscalar and out-of-order execution and extend the model to simultaneous multithreading (SMT), comparing instruction throughput and resource utilization.

Objectives

- Demonstrate issuing multiple instructions per cycle.
- Compare scalar and superscalar execution.
- Model a simple out-of-order scheduling benefit.
- Compare single-thread and SMT resource utilization.

Concept / Theory

A scalar processor issues one instruction per cycle in the simplified model. A superscalar processor can issue multiple independent instructions per cycle. Out-of-order execution allows ready instructions to execute when resources are available instead of waiting unnecessarily behind a stalled instruction. SMT allows instructions from multiple software threads to share the same execution resources.

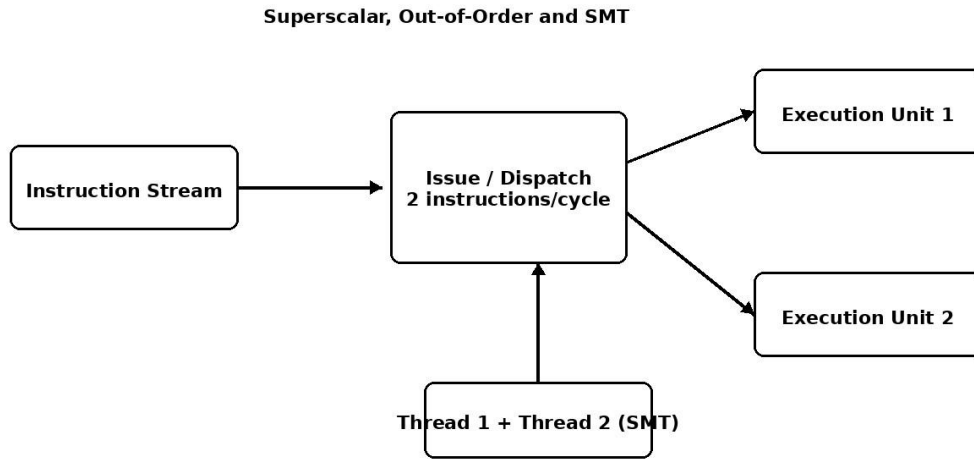


Figure 5. Conceptual superscalar and SMT datapath.

Experimental Design

The same instruction workload is evaluated under scalar and superscalar issue widths. A simple readiness model is used to demonstrate that independent instructions can execute while a dependent instruction is waiting. SMT is then modeled by interleaving two threads across the same execution width.

Extended Analysis – Out-of-Order Concept

For the final record, add a small instruction dependency example such as: I1 waits for memory, I2 is independent, I3 depends on I1. Explain that an out-of-order scheduler can execute I2 while I1 waits, improving functional-unit utilization without changing the architectural order in which results are committed.

Suggested Test Cases

Case	Issue Width	Threads	Purpose
C1	1	1	Scalar baseline
C2	2	1	Superscalar throughput
C3	2	2	SMT resource sharing
C4	2	2	Add dependency/waiting behavior for out-of-order discussion

Output / Evidence:

```
=====
PS C:\Users\SANDY\Downloads\CA CO3 AT1> python experiment5.py

=====
COMPUTER ARCHITECTURE - EXPERIMENT 5
SUPERSCALAR, OUT-OF-ORDER AND SMT PROCESSING
=====

=====
SCALAR IN-ORDER
=====
Cycle 1: Execute I1: ADD R1,R2,R3
Cycle 2: Execute I2: SUB R4,R5,R6
Cycle 3: Execute I3: MUL R7,R1,R8
Cycle 4: Execute I4: ADD R9,R4,R10
Cycle 5: Execute I5: SUB R11,R7,R12
Cycle 6: Execute I6: ADD R13,R9,R14
Cycle 7: Execute I7: MUL R15,R11,R16
Cycle 8: Execute I8: ADD R17,R13,R18

Scalar cycles = 8

=====
2-WAY SUPERSCALAR IN-ORDER
=====
Cycle 1: Execute I1: ADD R1,R2,R3 | Execute I2: SUB R4,R5,R6
Cycle 2: Execute I3: MUL R7,R1,R8 | Execute I4: ADD R9,R4,R10
Cycle 3: Execute I5: SUB R11,R7,R12 | Execute I6: ADD R13,R9,R14
Cycle 4: Execute I7: MUL R15,R11,R16 | Execute I8: ADD R17,R13,R18

Superscalar cycles = 4
```

```
=====
                        2-THREAD SMT EXECUTION
=====
Cycle 1: Execute T1-I1 | Execute T2-I1
Cycle 2: Execute T1-I2 | Execute T2-I2
Cycle 3: Execute T1-I3 | Execute T2-I3
Cycle 4: Execute T1-I4 | Execute T2-I4

SMT cycles = 4

=====
                        PERFORMANCE ANALYSIS
=====

Architecture          Cycles          IPC
-----
Scalar                 8             1.000
2-Way Superscalar     4             2.000
2-Way OoO              4             2.000
2-Thread SMT          4             2.000

Speedup over scalar:
2-Way Superscalar : 2.00x
2-Way OoO         : 2.00x
2-Thread SMT      : 2.00x

=====
                        VALIDATION
=====

✓ Scalar execution completed.
✓ Superscalar execution completed.
✓ Out-of-order execution completed.
✓ Two SMT threads executed concurrently.

VALIDATION : PASS
All four processor execution models completed successfully.
```

Results and Analysis

Metric	Scalar	2-Wide Superscalar	SMT
Cycles	Actual	Actual	Actual
IPC	Actual	Actual	Actual
Speedup	1.00×	Actual	Actual
Resource utilization	Actual	Actual	Actual

Analysis: Explain how issue width increases potential throughput, why actual speedup may be below the theoretical width when dependencies exist, and how

SMT can fill otherwise unused execution slots. Discuss the trade-off between performance and shared-resource contention.

Validation

Check that every issued instruction appears exactly once in the schedule and that the number of executed instructions equals the input workload. Compare the calculated IPC with instructions divided by measured cycles.

Result

Superscalar and SMT execution are successfully analyzed when parallel issue, throughput, IPC and resource utilization are measured and interpreted against the scalar baseline

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies(5)	Good understanding with proper integration of most concepts(4)	Basic understanding; limited or partial integration(3)	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution(6)	Correct implementation with minor issues(4)	Basic implementation with errors or inefficiencies(3)	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration(4)	Appropriate use of tools with correct execution(3)	Limited or inconsistent use of tools(2)	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results(3)	Good analysis with reasonable interpretation(2)	Basic analysis with limited insights(1)	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results(2)	Organized documentation with necessary details(1.5)	Basic documentation with missing details(1)	Poor or incomplete documentation